

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 5 REKURSIF



NAMA : RAFFI ADITIYA SAPUTRA

NIM : 109082500094

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Rekursif adalah teknik pemrograman dimana sebuah subprogram (fungsi/prosedur) memanggil dirinya sendiri untuk menyelesaikan suatu masalah. Secara sederhana rekursif dapat diartikan sebagai cara menyelesaikan suatu masalah dengan cara menyelesaikan sub-masalah yang identik dari masalah utama.

B. Guided

1. Membuat baris bilangan dari n hingga 1

Base-case: bilangan == 1

```
package main
import
"fmt"
func main() {
var n int
fmt.Scan(&n)
baris(n)
} func baris(bilangan
int) {    if bilangan
== 1 {
fmt.Println(1)
    } else {
fmt.Println(bilangan)
baris(bilangan - 1)
    }
}
```

Output :

```
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
al1.go"
5
5
4
3
2
1
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> |
```

Penjelasan:

Program tersebut merupakan penerapan rekursif yang terjadi ketika sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan masalah yang lebih kecil. Terdapat Struktur Fungsi **baris**:

Fungsi ini bertujuan untuk mencetak angka secara menurun dari n sampai 1. Terdapat dua bagian:

- Base case: **if `bilangan == 1`**. Tanpa program tersebut program akan berjalan selamanya tanpa henti.
- Recursive Step: **else `{baris(bilangan – 1)}`**. Jika angka belum mencapai 1, program mencetak angka saat ini, lalu memanggil fungsi **baris** dengan nilai yang dikurangi 1.

2. Menghitung hasil penjumlahan 1 hingga n

Base-case: `n == 1`

```
package main
import
"fmt"
func main() {    var n int
fmt.Scan(&n)
fmt.Println(penjumlahan(n))
} func penjumlahan(n int) int {
if n == 1 {      return 1
} else {         return n +
penjumlahan(n-1)
}
}
```

Output :

```
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
a12.go"
5
15
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
a12.go"
3
6
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> █
```

Penjelasan:

Program ini juga menggunakan rekursif, tujuannya untuk menghitung jumlah total (sum) dari deret angka, mulai dari n hingga 1.

- Cara kerja fungsi **Penjumlahan**
Fungsi ini sebagai penambahan sampai mencapai angka terkecil (1).
- Base Case (kondisi berhenti): **if n == 1 { return 1 }**. Jika n sudah mencapai 1, fungsi tidak akan memanggil dirinya lagi.
- **Return n + penjumlahan(n - 1)**. Hasil penjumlahan n adalah n akan ditambah dengan hasil penjumlahan angka sebelum n.

3. Mencari dua pangkat n atau 2^n Base-case: $n == 0$

```
package main

import "fmt" func main()
{
    var n int
    fmt.Scan(&n)
    fmt.Println(pangkat(n))
}

func pangkat(n int) int {
    if n == 0 {          return 1
    } else {             return 2 *
    pangkat(n-1)
    }
}
```

Output :

```
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
a13.go"
5
32
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
a13.go"
0
1
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> |
```

Penjelasan:

Program ini menghitung 2 pangkat N (2^n) secara rekursif. **Fungsi main()** menerima input N dari user, kemudian memanggil fungsi **pangkat(n)** dan mencetak hasilnya. Fungsi

mengalikan 2 sebanyak N kali hingga mencapai N = 0, kemudian hasilnya dikembalikan bertahap ke atas (backward).

C. Unguided

1. Soal Unguided 1

Deret fibonacci adalah sebuah deret dengan nilai suku ke-0 dan ke-1 adalah 0 dan 1, dan nilai suku ke-n selanjutnya adalah hasil penjumlahan dua suku sebelumnya. Secara umum dapat diformulasikan $S_n = S_{n-1} + S_{n-2}$. Berikut ini adalah contoh nilai deret fibonacci hingga suku ke-10. Buatlah program yang mengimplementasikan fungsi rekursif pada deret fibonacci tersebut.

n	0	1	2	3	4	5	6	7	8	9	10
S^n	0	1	1	2	3	5	8	13	21	34	55

Jawaban :

```
package main

import "fmt"

func fibonacci(n int) int {
    if n == 0 {
        return 0
    } else if n == 1 {
        return 1
    }
    return fibonacci(n-1) + fibonacci(n-2)
}

func main() {
    fmt.Println("Deret Fibonacci hingga suku ke-10: ")
    for i := 0; i <= 10; i++ {
        fmt.Printf("%-5d %-5d\n", i, fibonacci(i))
    }
}
```

Output :

```

PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
soal1.go
Deret Fibonacci hingga suku ke-10:
n      Sn
0      0
1      1
2      1
3      2
4      3
5      5
6      8
7      13
8      21
9      34
10     55
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA>

```

Penjelasan :

Program ini digunakan untuk menghitung dan menampilkan deret Fibonacci hingga suku ke-10 menggunakan metode rekursif. Deret fibonacci adalah urutan angka dimana setiap angka adalah hasil penjumlahan dari dua angka sebelumnya.

1) Fungsi Rekursif **Fibonacci**

Fungsi ini adalah otak utama dari program untuk menentukan nilai suku ke-n dengan beberapa aturan:

- Dua kondisi Dasar (Base case)

Program akan mengecek apakah n adalah 0 atau 1. Jika $n == 0$, akan mengembalikan nilai 0. Jika $n == 1$, akan mengembalikan 1. ○ Jika n lebih besar dari 1, fungsi akan memanggil dirinya sendiri dua kali secara bersamaan.

2. Soal Unguided 2

Buatlah program yang mengimplementasikan rekursif untuk menampilkan faktor bilangan dari suatu N, atau bilangan yang apa saja yang habis membagi N. Masukan terdiri dari sebuah bilangan bulat positif N. Keluaran terdiri dari barisan bilangan yang menjadi faktor dari N (terurut dari 1 hingga N ya).

Contoh masukandankeluaran:

No	Masukan	Keluaran
1	5	1 5
2	12	1 2 3 4 6 12

Jawaban :

```

package main
import
"fmt"
func cariFaktor(i, n
int) {      if i > n {
return
        }      if n%i == 0 {
fmt.Printf("%d ", i)
        }
cariFaktor(i+1, n)
} func main() {      var n int
fmt.Print("Masukkan nilai N: ")
fmt.Scan(&n)
        fmt.Print("Keluaran:
")      cariFaktor(1, n)
fmt.Println()
}

```

Output :

```

PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
soal2.go
Masukkan nilai N: 5
Keluaran: 1 5
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
soal2.go
Masukkan nilai N: 12
Keluaran: 1 2 3 4 6 12
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA>

```

Penjelasan :

Program dimulai dengan meminta user memasukkan sebuah nilai ke dalam variabel `n` melalui perintah **fmt.Scan**. Program memanggil fungsi **cariFaktor(1, n)**. dimulai dari angka 1 karena 1 adalah faktor terkecil dari setiap bilangan bulat positif. **If i > n**. program akan berhenti memanggil dirinya sendiri jika angka pembagi (`i`) lebih besar dari angka yang dicari (`n`), untuk mencengah program berjalan selamanya tanpa henti.

3. Soal Unguided 3

Buatlah program yang mengimplementasikan rekursif untuk menampilkan barisan bilangan ganjil. Masukan terdiri dari sebuah bilangan bulat positif N.

Keluaran terdiri dari barisan bilangan ganjil dari 1 hingga N. **Contoh**

masukan dan keluaran:

No	Masukan	Keluaran
1	5	1 3 5
2	20	1 3 5 7 9 11 13 15 17 19

Jawaban :

```
package main
import
"fmt"

func cetakBilanganGanjil(i, n int)
{
    if i > n {
        return
    }
    if i%2 != 0 {
        fmt.Printf("%d ", i)
    }

    cetakBilanganGanjil(i+1, n)
}

func main() {
    var n int
    fmt.Print("Masukkan nilai N: ")
    fmt.Scan(&n)
    fmt.Print("Keluaran: ")
}

cetakBilanganGanjil(1, n)
fmt.Println()
}
```

Output :

```
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
soal3.go
Masukkan nilai N: 5
Keluaran: 1 3 5
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
soal3.go
Masukkan nilai N: 20
Keluaran: 1 3 5 7 9 11 13 15 17 19
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA>
```

Penjelasan :

Program meminta pengguna memasukkan sebuah angka bilangan bulat positif yang disimpan ke dalam variabel **n**. program memanggil fungsi **cetakBilanganGanjil(1, n)**. Angka 1 sebagai titik pencarian karena merupakan bilangan ganjil pertama. Baris **if i > n** memastikan program berhenti bekerja ketika angka yang diperiksa (**i**) sudah melewati batas angka yang sudah ditentukan oleh (**n**). Program menggunakan operasi modulo (**i%2 != 0**). Jika suatu angka dibagi 2, maka angka tersebut dinyatakan sebagai bilangan ganjil.

D. Kesimpulan

Rekursif adalah sebuah teknik pemrograman dimana sebuah sub-program memanggil dirinya sendiri. Teknik ini digunakan untuk menyelesaikan masalah dengan cara memecahnya menjadi sub-masalah yang lebih kecil namun identik dengan masalah utama. Rekursif dapat digunakan sebagai alternatif untuk menggantikan struktur kontrol perulangan (looping). Agar sebuah fungsi rekursif dapat berjalan dengan benar dan tidak menyebabkan perulangan tanpa henti (infinite loop), diperlukan dua komponen **Base-case(Basis)** dan **Recursive-case**.